

Introduction

The aim of this screencast is to demonstrate your project and your understanding of its workings. Your goal is to:

- Explain the high-level architecture of the system (the various layers/components and how they interact)
- Explain how the underlying code works while demonstrating the system running

Team members should outline the overall system design together, followed by each demonstrating and explaining their own contributions to the system. Where functionality was developed by both team members, both members should participate in outlining the way that functionality works overall.

Recommended Structure [times are suggested/upper limits, not requirements!]:

- 1) Introduction (~1 min):
 - a. State your names
 - b. Each student should briefly list the general areas they handled
 - c. If you used any external libraries, identify these and state what they were used for
- 2) System overview (this should cover the entire system, not just your own contribution) (2-3 mins):
 - a. Outline components involved in system
 - b. Outline how components interact - I suggest illustrating this with a simple diagram showing the components and their control flow/interaction points, as it's easier to talk around a diagram
 - i. An example of the type of diagram you could use is the one in the slides showing the structure of the MVC pattern, but with more specifics (i.e. all classes within a layer named and shown individually, rather than showing the whole layer as one box)
- 3) Project demonstration (5-10 mins):
 - a. Show the various user stories/system features on which you worked
 - i. Note: Where there is functionality "in the way" of getting to your logic (e.g. your teammate did login/register and you did view all books, but you can't view the books until you've logged in), state that your teammate handled that part, give a one-line explanation and continue through
 - b. Show valid and incorrect use-cases. Some examples include:
 - i. Login: show failed attempts as well as a successful one.
 - ii. Borrow: show a loan successfully being created as well as where a loan couldn't be created.
 - iii. Return: show an attempt to return a loan a user does not have.
 - c. Explain as you demonstrate
 - i. As you're using the system, give a high-level explanation of what's happening in the code and what parts are responsible. Toggle between showing the system interface and the code being executed, gesturing with the cursor to show which sections of code are involved.
 - ii. Example of demonstrating register user story:
 1. "When I click register with a confirmation password that does not match the first version *<showing interface>*
 2. The register view *<showing view code>* calls the XXX controller
 3. The XXX controller calls the XXX model *<showing controller code>*.
 4. The XXX model validates the information and returns an error *<showing model code>* back to the XXX controller.
 5. The XX controller returns the error back to the view *<showing controller, then view>*, which triggers a change in the view's display"
 - iii. Note: You do not need to give a low-level walkthrough/reading of the code!
- 4) Key areas (2-3 mins):
 - a. Discuss any key aspects/areas you contributed that are core to the system functionality, showcase a clever technique/approach (i.e. something you're proud of!) or demonstrate an important design decision (something that shaped the way the project developed)
 - b. Explain **what** it does and **why** you developed it in this way
- 5) Challenges (1-2 mins):
 - a. Give a brief description of any challenges you encountered and how you approached/solved them
- 6) Retrospective (1-2 mins):
 - a. Briefly outline what you learned from this development cycle
 - b. Explain what, if any, changes you would have made knowing what you know now.